Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech - ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026 - 27	**Semester:**	3/4 Sem - I
Name of the Student:	T. Mahesh.	**Roll Number:**	A24126552274
Name of the Lab:	Full Stack - Lab	**Timings:**	

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	1					13		
8/7	2					14		
15/7	3					13		
22/7	4					13		
29/7	5					14		
5/8	6					14		
12/8	7					13		
19/8	8					13		
2/9	9					13		
9/9	10					14		
9/9	11					14		
9/	12							
	13							

Name of the Faculty: Ch. Sruthi

Faculty Signature: Sruthi

Introduction to HTML and CSS

→ HTML (Hyper text markup language) is the Standard markup language used to create and Structure content on web pages. It defines the basic Structure of a wb page using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (Cascading Style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, background, spacing, borders, alignment, dimensions, and layouts. CSS also supports responsive design, allowing web pages to adapt to different screen sizes and devices. CSS can be applied using inline, internal or external stylesheets.

work together to develop webpages, Where HTML provides the structure and content while CSS provides the styling layout, visual presentation The combination of both forms the foundation of modern web development and enables the creation of organized, attractive, and user-friendly web interfaces.

Difference between MERN Stack and FULL Stack :

→ MERN STACK is a javascript-based technology Stack used for developing modern web applications.

MERN stands for MongoDB, Express.js, React.js, Node.js, MongoDB is used for database management, Express.js handles the backend framework and Apis, React.js is used to develop frontend User interface, and Node.js provides the server side runtime environment

→ Full Stack Development refers to the development of both the frontend and backend of a web application along with database integration.

A full stack developer works with technologies used for designing works with the user interface. implementing server-side logic, managing databases.

HTML Components :

1. HTML Tags - Define the structure and elements of a webpages.
2. Headings - Used to Display titles and headings using <h1> to <h6>
3. paragraphs - Used to display text using the <p> tag
4. Links — Used to navigate between webpages using <a> tag.
5. Images - Used to display images using <image> tag
6. Lists - Used to create ordered and unordered lists using <ol>, <ul>,
7. Tables - Used to organize data in rows and columns.
8. Forms - Used to collect user input using elements such as text boxes, buttons and checkboxes.
9. Div and span - Used to group and organize webpages content
10. Semantic Elements - Elements such as <header>, <nav>, <selection>, <article>, <footer>, provide meaningful webpage structure.

Css Components:

1. Selectors - Select HTML elements to apply styles
2. properties - Define the visual characteristic of elements
3. Values - Specify the setting of a css properties
4. Colors - Define text, background, border colors
5. Fonts - Control font family, size style and weight
6. Backgrounds - set background colors and images.
7. Borders - Define the border style, width and radius.
8. Margin - Controls the space outside an element
9. padding - Controls the space inside element
10. Layout - Controls the arrangement of elements Using flexbox, Grid and positioning

HTML VS HTML5

HTML	HTML5
older version of HTML	Latest Major version of HTML
Limited multimedia support	Support audio and video
Uses external plugins for multimedia	No external plugs needed
Has fewer Semantic tags	provides semantic tags
No <canvas> element	supports <canvas>
Limited web storage	supports local Storage
Complex DocType declaration	Simple <!DocType HTML>

Css Vs Css3

Css	Css 3
Earlier version of css	Advanced version of css
Basic styling features	provides advanced styling features
Limited animations	Supports animations and transitions.
No flexible layout systems	Supports Flexbox and Grid
Limited visual effects	supports shadows, gradients, effects
No media queries	supports media queries
Uses a single specification	divided into different modules.

(i) Design an HTML code using audio tags

Code:

```
<!DOCTYPE HTML>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My Favorite Song </h2>
    <audio controls>
        <source src="song.mp3" type="audio/
        Your browser does not support the
            audio tag
    </audio>
</body>
</html>
```

O/p: Displays an audio player with play, pause and volume controls.

Steps:

Create a new HTML file
Add the basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with controls attribute
Specify the audio file using <source> tag
Save and open the HTML file in a web browser play the audio using displayed controls

(ii) Design a HTML code using video tag

Code:

```
<!Doc Type HTML>
<html>
<head>
    <title> Vidka Example </title>
</head>
<body>
    <h2> My video </h2>
    <video width="400" controls>
        <source src="video.mp4" type=
            "video/mp4">
    </video>
</body>
</html>
```

O/p: Displays a video player with play, pause Volume, and Full Screen controls.

Steps:

Create a new HTML file
Add a basic HTML structure
Add a heading using the <h2> tag
Insert the <video> tag with controls attributes
Specify the video file using <source> tag
Save the file and open it in web browser
play the video using displayed controls.

Write a program using JavaScript (arrays, functi

Code:

```
let number = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];
function displayArray (arr) {
    Console. log ("Array Elements: ");
    for (let i = 0; i< arr. length ; i++) {
        Console .log (arr[i]);
    }
}
function findSum (arr) {
    let sum = 0;
    for (let i = 0; i<arr . length ; i++){
        sum + = arr[i];
    }
    return sum;
}
```

Experiment - 4

4. Develop dynamic webpage using java script

Program:

```
<!Doctype html>
<html>
<head>
    <title > Dynamic webpage <title>
    <style>
        body {
            text -align : center;
            font - family : Arial;
            margin - top : 100 px:
        }
    button {
        padding : 10px 20px ;
        font _size : 16 px;
        cursor : pointer
    }
</style>
</head>
<body>
    <h1 id = "heading "> welcome to
        my webpage </h1>
<p id = "message"> click the button to
    change this webpage </p>
```

```
<button onclick = "change page()"> Click me
                </button>
<script>
    function change page.(){
    document . get element By id ("heading"),
            innerHTML = "Welcome to java Scri
    document . get element By id ("Message").
            inner HTML = "The webpage has be
    changed dynamically. ";
document . body Style . background color =
                "light blue";
    }
</script>
</body>
</html>
```

Output:

Welcome to my webpage

click the button to change the webpage

[Click Me]

- When the user clicks "click me", the javascript function change page() is called, then the heading dynamically changes and background also changes.

After clicking the button.

Welcome to . Java Script !

The webpage has been changed dynamically

[Click Me]

Dynamic Webpage

File C:/Users/Tejaswari/Desktop/index.html

Welcome to My Webpage

Click the button to change this webpage.

Click Me

Before clicking the button

Dynamic Webpage

File C:/Users/Tejaswari/Desktop/index.html

Welcome to JavaScript!

The webpage has been changed dynamically.

Click Me

After clicking the button

Experiment -05

(5) Implement modules in Node JS.

- In Node js, you can implement modules using two distinct systems: Common Js (CJS) which is traditional default, and ESModules (ESM), the modern javaScript Standard.

1. The Common Js Method (defaults):

Commo Js uses module.exports to share Code and require() to impat it. This System loads modules Synchronously.

a. Create the module (math.js):

```
Const add = (a, b) => a+b;
Const Subtract = (a, b) => a-b;
module.exports = {
    add,
    Subtract
};
```

b. Use the module (app.js):

```
Const math = require('./math');
Console.log (math.add (10,5));
Console.log (math.subtract (10,5));
```

output: 15
5

2. The Fs Modules Method (Modern Standard):

Es Modules use export to share code and import to use it To make Node.js recognize Es modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a. Create the module (calculator.js):

```
Expot const multiply = (a, b) => a * b
expot const divide = (a, b) => a / b

Const calculatorName = "supercalc";
Expot default calculatorName;
```

b. Use the module (main.js):

```
import calculator, {multiply, divide}
                from './calculator.js';

console.log (calculatorName);
console.log (multiply (4, 2));
console.log (divide (4, 2));
```

output: Supercalc
8
2

3. Using Built-in core modules:

Node.js also comes equipped with dozens of built-in core modules (like fs for file systems or path for file routing) that don't need installation.
we will load them by module names:

```
// Common JS style
Const fs = require ('fs');

// Es modules style
import fs from 'fs';
```

Here are some of the most commonly used ones;

1. fs = file system operations
2. Http - HTTP server and client
3. path - file path utilities
4. os - operating system utilities
5. events - Events handling
6. util - utility functions
7. stream - Stream handling
8. Crypto - Cryptographic functions
9. url - URL parsing

10. query String – URL query string for handling

Example: Simple HTTP Server

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.end("Hello world!");
}).listen(8080);
```

Output:- Hello world!

Experiment – 06

6. Develop mini application using express and Node Js:

Simple mini application using node Js + Express Js.

Student Management System.

1. Add a Student
2. View all Students
3. delete a Student
4. Uses Express.js routes
5. Simple HTML interface

project Structure:

```
Student
  ├── server.js
  └── public
        └── index.html
```

Server.js

```
Const express = require("Express");
Const app = express();
Const PORT = 3000;
  let students = [];
app.use(express.json());
```

```
app.use (express.static ("public"));
app.get ("/students", (req, res) => {
        res.json (students);
});

// Add
app post ("/students", (req, res) => {
        Const student = {
            id: students.length +1,
            name: req.body.name,
            Course: req.body.Course
        };
        Students.Push (student);
        res.json ({message: "student added
                    Succesfully",
                    student: student});
});

// Delete
app.delete ("/students/:id", (req, res) => {
        const id = parseInt (req.parents.id);
        Students = students.filter (student =>
                    Student.id !== id);
        res.json ({
            message: "student deleted Succesfully"
```

```
// Start Server
app.listen (PORT, () => {
        Console.log ("Server running at http://local
                    host:${PORT}");
});
```

public/index.html:

```
<!DocTYPE HTML>
<html>
<head>
        <title> Student management </title>
</head>
<body>
        <h1> Student management system </h1>
        <input type = "text" id = "name"
        placeholder = "Enter Student name">
        <input type = "text" id = "Course"
        placeholder = "Enter course">
        <button onclick = "add student ()">
                Add Student </button>
        <h2> Student list </h2>
        <ul id = "Student list"> </ul>
<script>
        function add student() {
            let name = document.getElementById
                    ("name").value;
```

```
let course = document.getElementById
             ("course")
             value;

fetch ("/students",{
        method : "POST",
        headers:{
            "Content-Type" : "application/json"
        body: JSON.stringify({
                name: name,
                course: course
            })
        })
    .then (response => response.json
    .then (data =>{
                alert (data.messag
document.getElementById ("name").value = "";
document.getElementById ("course").value = "";
        displayStudents ();
        });
    }
function displayStudents (){
    fetch ("/students").then (response => response.
                json())
    .then (students => {
        let list = document.getElementById
                ("student list");
        list.innerHTML = "";
students.forEach (student => { let li =
        document.createElement ("li");
        li.innerHTML = student.id + "-" +
                = student.name + "-" +
                = student.course + "<
                button
onclick = 'deleteStudent (" + student.id + ")' >
                Delete </button>";
        list.appendChild (li);
            });
        });
    }
    function deleteStudent (id){
        fetch ("/students/" + id, {method: "DELETE"})
    .then (response => response.json())
    .then (data => {
            alert (data.message);
            displayStudent ();
        });
    }
```

displayStudents();

</script>

</body>

</html>

Output:

Student Management System

Enter Student name

Enter Course

Add Student

Student list :-

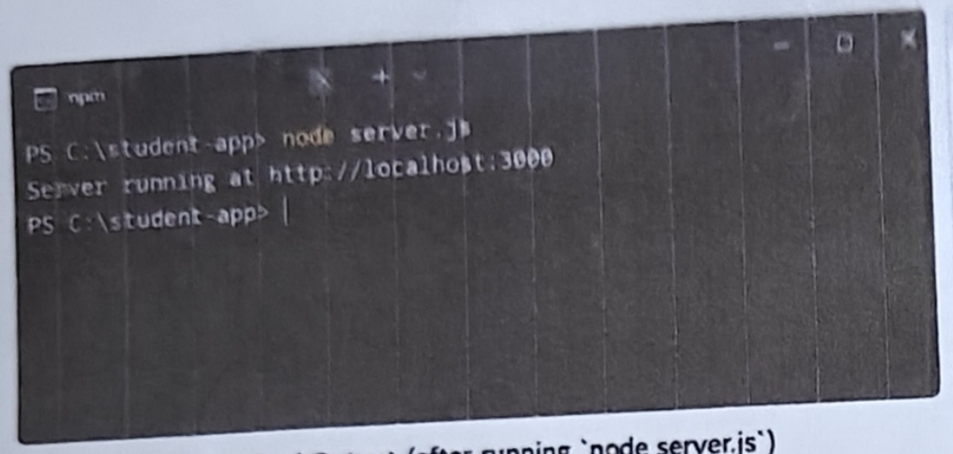
1. Manohar - CSM [delete]

2. Manoj - CSE [delete]

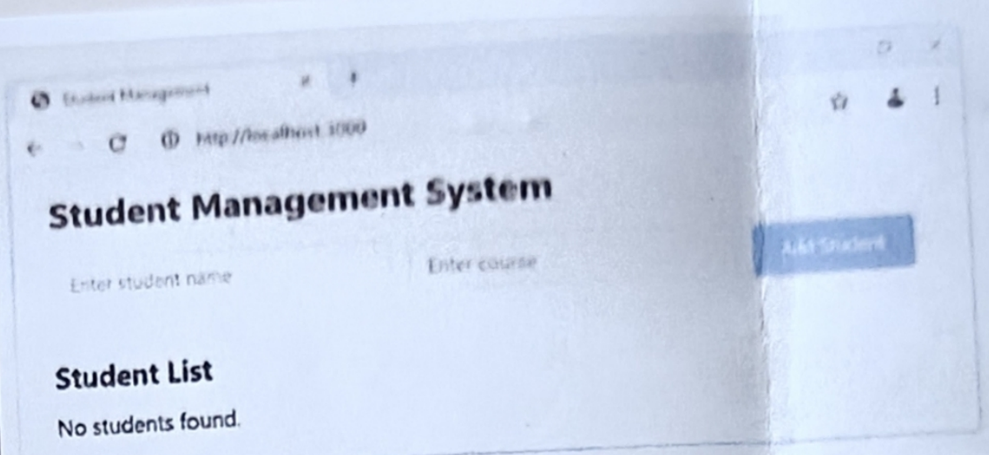
- The Express.js Server is Succesfully Started at

http://localhost:3000

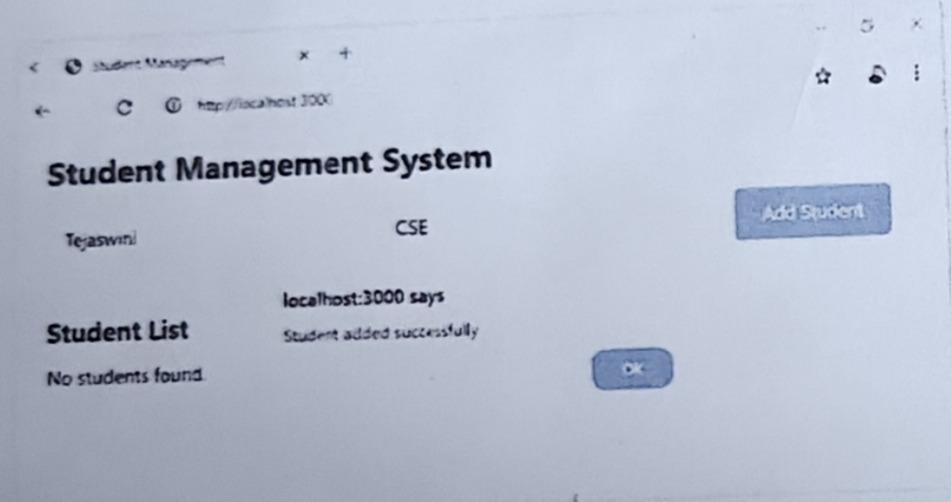
- The browser displays the Student Management System with fields.



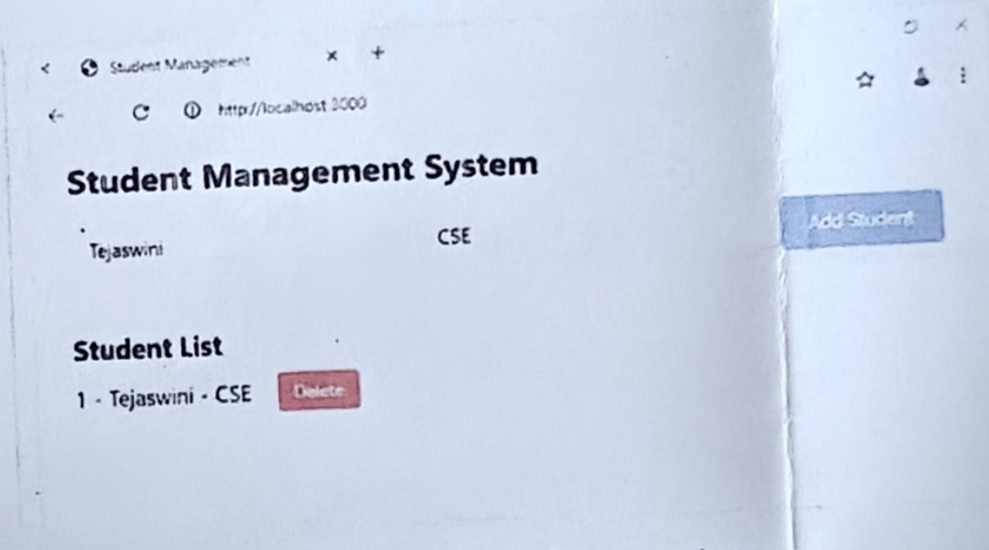
1. Terminal Output (after running `node server.js`)



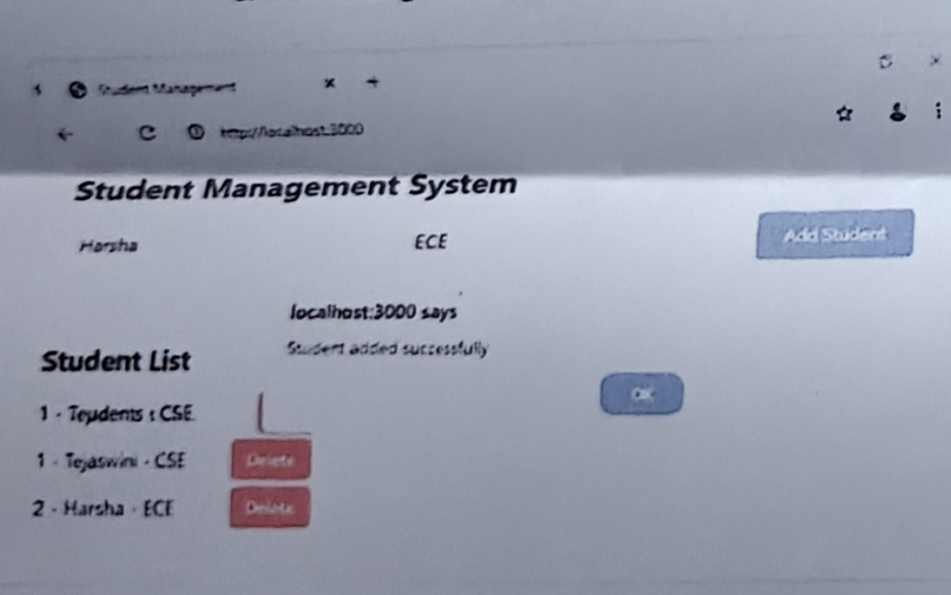
2. Initial Output in Browser (no students)



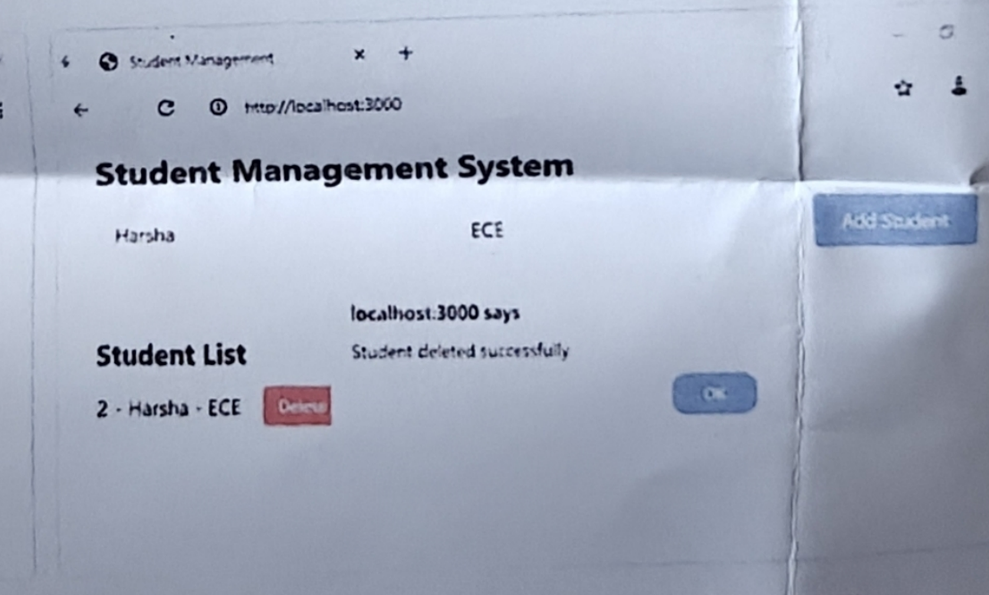
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment - 07

1. Create a functional component and class Component using react

Aim: To develop a react application using functional components and class components to display student details, manage state. handle button clicks and dynamically update data on a webpage.

Functional Components:

A functional Component is a java script function that return JSX to display the users interface. It uses react Hooks such as restate to manage state and update the webpage dynamicalley.

program:

```
import React, {useState} from
"react";
function student function() {
    Const [name, Set name] = use State ("girish");
    Const [Course, Set Course] = use State ("CSM");
    Const [Count, Set Count] = Use State (0);
Const Change Name = () => {
```

```
    Set Name ("giri");
};
Const ChangeCourse = () => {
    Set Course ("CSE");
};
Const increment Count = () => {
    Set count (Count + 1);
};
return (
  < div Style ={{
      border : "2 px "solid blue";
      padding : "20 px",
      margin : "20 px",
      border Radios : "10 px";
      text Align : "center"
  }}>
  <h1> Student Details </h1>
  <h2> Functional Component </h2>
  <p> <b> Name : </b> {name} </p>
  <p> <b> Course : </b> {course} </p>
  <p> <b> Button clicks : </b> {Count} </p>
  <button on click ={ change Name }>
      Change Name </button>
  <button on click ={ change Course } Change
      Course </button>
  <button on click ={increment Count} >
      Increment
  </button>
  </div>
  );
```

Output: when the user clicks the buttons, the name, Course and Count change dynamically without reloading the webpage.

* Class Component:

A class component is an ES6 Java Script class that extends React Component. It uses the render() method to return JSX and manages state using this.state and this.setState()

Program:

```
import React, {Component} from 'react';
class Student class extends Component{
    Constructor (props){
        Super (props);
        this.State = {
            name : "Girish";
            Course : "Computer Science",
            Count : 0
        };
    }
Change Name = () => {
    this.setState ( {
            name : "giri"
    });
};
Change Course = () =>{
    this.setState ({
        Course : "CSH"
    });
};
increment Count = () => {
    this.setState ((prev State) => ({
        Count : prev State.Count + 1
    }));
};
render.() {
    return (
    <div Style = {{ border : "2px Solid green"
        padding : "20px";
        margin : "20 px",
        border Radius : "10px",
        text Align : "Center "}}>
    <h1> Student Details </h1>
    <h2> Class Component. </h2>
    <p> <b> Name : </b> {this.State.name}</p>
    <p><b> Course : </b> {this.State Course}</p>
    <p><b> Button clicks: </b> {this.State Count}</p>
    <button onClick ={this Change Name} >
        Change Name </button>
```

<button onclick={this.changeCourse}>
Change Course </button>

Experiment - 8

React App

localhost:3000

React Components

Student Details

Functional Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

```
<button onclick={this.changeCourse}>
        Change Course </button>

<button onclick={incrementCount}>

<button on click={this.incrementCount}>
    Increment
    </button>
  </div>
}
```

Output:

Experiment - 8

8. Implement Http methods using JSON Server

Aim: To implement HTTP methods such as GET, POST PUT, DELETE using JSON Server to perform CRUD. (Create, Read, update, and Delete) operations on Studet resects.

Theory: JSON Server is a tool used to create a fake REST API Using a JSON file it allows us to perform HTTP operations without developing a backend Server from scratch.

HTTP methods are used to Communicate between Client and a Server.

* GET - Retreives data from the Server
* POST - Adds new data to the Server
* PUT - Replaces an existing record
* PATCH - updates selected fields of a Record.
* DELETE - Removes a record from the Server

a) Create a JSON Server

Step 1: Initialize the project

Open the vscode terminal and execute the following Commands

Mkdir Json Server

cd Json Server

npm init -y

npm install json server

Step 2: Create db-json

Create a file named db.json and add the following Student data.

```
{
  "students": [
    {"id": "1",
     "name": "simba",
     "course": "CSM",
     "Marks": 90
    },
  {
    "id": "2"
    "name": "Girish",
    "course": "CSE",
    "Marks": 90
  },
  {
    "id": "3",
    "name": "Nani",
    "course": "IT",
    "Marks": "88,
```

```
    }
  ]
}
```

Step 3: Start the Json Server

For the current Json server version execute

npx json server db.json

The server start at

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students.

2. Implementation of HTTP Methods:

a. GET METHOD: The GET method is used to retreive student records from Json Server

Code:

```
fetch("http://localhost:3000/students")
then(response => response.json())
then(data => {
    console.log(data);
}).
catch(error => {
    console.log(error);
});
```

Output:

b. POST METHOD: The post method is used to insert a new student record into the Server.

Code:

```
fetch ("http://localhost:3000/students",
  {method : 'post',
   headers :{
   'Content-Type': 'application/json'
  },
body : JSON.stringify ({
    name : "Girish"
    Course; "CSE"
    marks : 92
  })
})
```

```
.then (response => response.json())
.then (data => {
    Console.log (data);
})
  .catch (error => {
    Console.log (error);
  });
```

Output:

C. PUT METHOD: The PUT method is used to replace an existing student record with updated data

Code: fetch ("http://localhost:3000/students/1")

```
{
 method : PUT
 headers : {
  'Content-Type': "application/json".
 },
```

```
body : JSON.stringify ({
    name : "Simba",
    Course : "AIML",
    Marks : 95
  })
})
.then (response => response.json())
.then (data => {
        Console.log (data);

}).
.catch (error => {
        Console.log (error);
});
```

output:

D. PATCH METHOD: The patch method is used to update only selected fields of an existing Student record.

Code:

```
fetch.("http://localhost:3000/students/2")
    { method : 'PATCH',
      headers : {
        'content-Type : "application/json"
    },
    body : JSON.stringify.({
        marks : 96
    })
})
.then (response => response.json())
.then (data => {
  Console.log (data);
})
.catch (error => {
   Console.log (error);
});
```

output:

E. DELETE METHOD: The DELETE method is used to remove an existing student record from the server.

Code: fetch ("http://localhost:3000/students/3", {
method: 'DELETE'
})
then (response => {
if (response.ok){
Console.log ("student deleted successfully");
}
})
catch (error => {
Console.log (error)
})

output:

* After executing the POST, PUT, PATCH and DELETE operations in sequence the remaining student will be

output:

Thus, HTTP methods GET, POST, PUT, PATCH and DELET were successfully implemented using JSON Server. The CRUD operations were performed on student records.

The data was retrieved, inserted, update and delete using HTTP requests.

1. GET All Students (http://localhost:3000/students)

localhost:3000/students

localhost:3000/students


```
[
  {
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
  }
]
```


2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234


```
{
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
}
```


3. POST (Create Student) – Using Postman

localhost:3000/students

localhost:3000/students


```
{
    "name": "Tejaswari",
    "course": "AIML",
    "marks": 95,
    "_id": "68dcf9b7e4c9e8e1f3a6c5678",
    "__v": 0
}
```


Status: 201 Created Time: 48 ms Size: 160 B

4. PUT (Update Student) – Using Postman

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234


```
{
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "AIML",
    "marks": 96,
    "__v": 0
}
```


Status: 200 OK Time: 62 ms Size: 165 B

5. DELETE (Remove Student) – Using Postman

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234


```
{
    "message": "Student deleted successfully"
}
```


Status: 200 OK Time: 45 ms Size: 46 B

6. GET All Students (After Deletion)

localhost:3000/students

localhost:3000/students


```
[]
```


Status: 200 OK Time: 28 ms

Experiment - 9

9. Develop CRUD operations using Mongo. DB.

Aim: To develop a wheb application using Node.js, Express.js and Mongo.DB to perfom CRUD operation (Create, Read, update delete) on Student records.

Theory: Mongo DB is a .Nosql database that Stores data in flexible, JSON like documents, CRUD operations are used to manage data in a database.

Mongoose is an Object data Modeling (ODM) library for Mongo.DB and Node.js It helps define Schemas, interact with Collections and perform database operations.

1. Installation and project Setup.

```
mkdir Mongo DB. CRUD
Cd Mongo DB - CRUD
npm init -y
npm install express mongoose
```

2. program Server.js

```
Const express = require ('express').
Const mongoose = require (mongoose);
Const app = express ();
Const PORT = 3000;
app.use (express.json ())
Mongoose.connect ('mongodb://12:0:0:1:
                    27017/College')
  .then (() => Console.log ("Mongodb Connected
  .Catch (err => Console.log (err));
// Create Student
Const StudentSchema = new
Mongoose.Schema ({
    name : {
      type : String,
      required : true
    },
    Course: {
      type: String,
      required : true
    },
    marks : {
      type : number,
      required : true
    }
```

```
});
// Create Student model
Const student =
    mongoose.model('Student', StudentSchema);

// Create Add
app.post('/students', async (req, res) => {
    try{
      Const Student = new student (req.body);
      Const SavedStudent = await Student.save();
      res.Status(201).json(SavedStudent);
    } Catch (error){
        res.Status(400).json({message, error:
                                message});
    }
});
// Read : Get all Students
app.get('/Students', async (req, res) => {
    try{
    Const Students = await Student.find();
    res.Status(500).json({message: error.
                          message});
    }
});
```

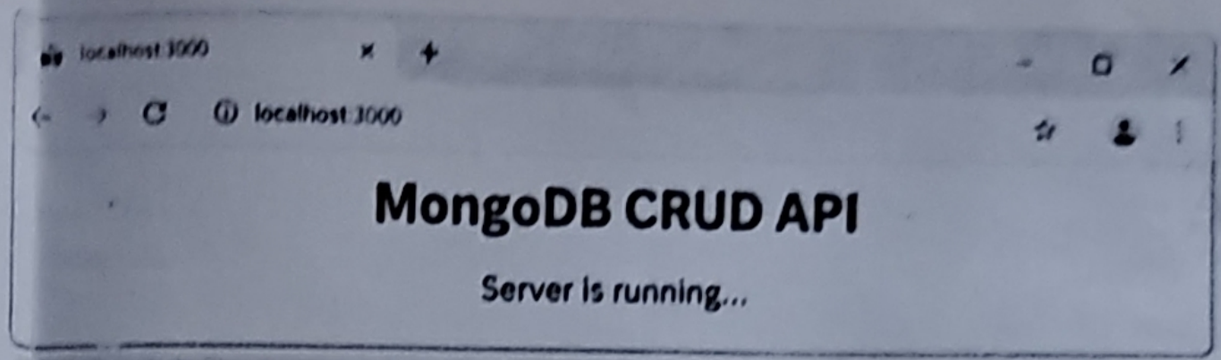
```
// Read : Get Student by ID
app.get('/Students/:id', async (req, res)
    => {
        try{
        Const Student = await
        Student.findById(req.params.)
        if(!Student){
        return res.Status(404).json({
            message: "Student not found"
        });
        }
      res.Status(200).json(Student);
} Catch (error){
    res.Status(400).json({message:
                          error.message});
    }
});

// Update.
app.put('/students/:id', async (req, res) => {
        try{
        Const Student = await
        Student.findByIdandUpdate(
            req.params.id
            req.body,
```

```
}, {new : true, runValidators : true});

if (!student){
    return res.status(404).json({
        message : "Student not found"
    });
}

res.status(200).json(students);
} catch (error){
res.status(400).json({message : error.message});
}
});

// Delete
app.delete('/students/:id', async (req, res)
    => {
    try{
    const student = await Student.
        findByIdAndDelete(
            req.params.id
        );
    if(!student){
        return res.status(404).json({
            message : "Student not found"
        });
    }
    res.status(200).json({
        message : "Student deleted Successfully"
    });
    catch (error){
    res.status(400).json({message :
                error.message});
    }
});
```

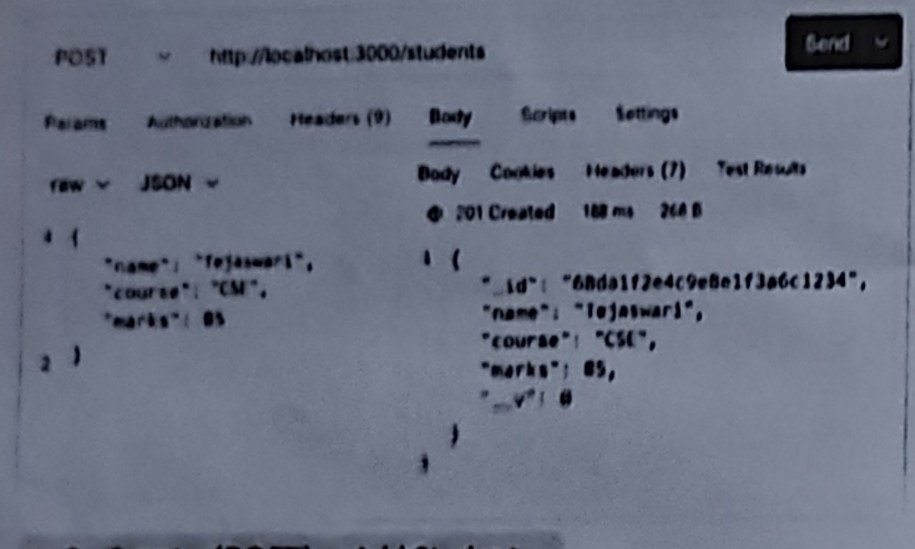
Output:

```
PS C:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
```

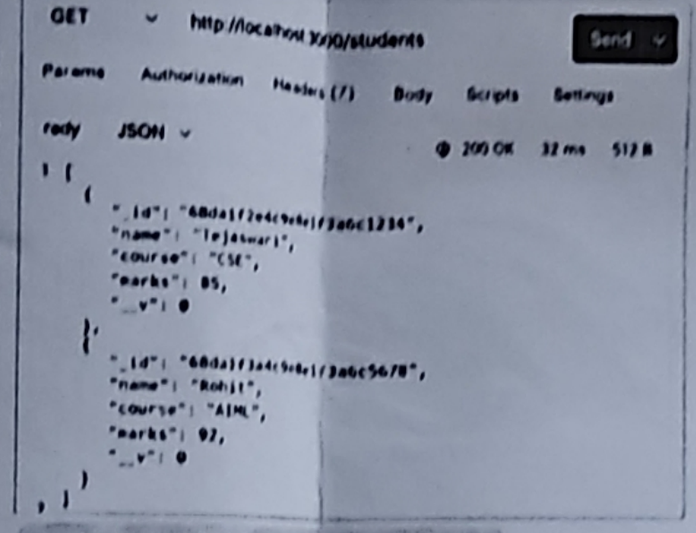
1. Terminal Output



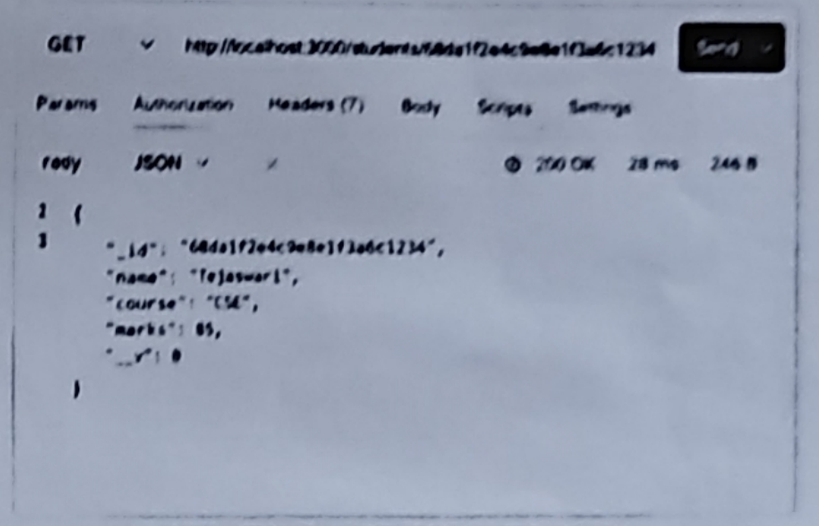
2. Chrome Browser (Home Page)



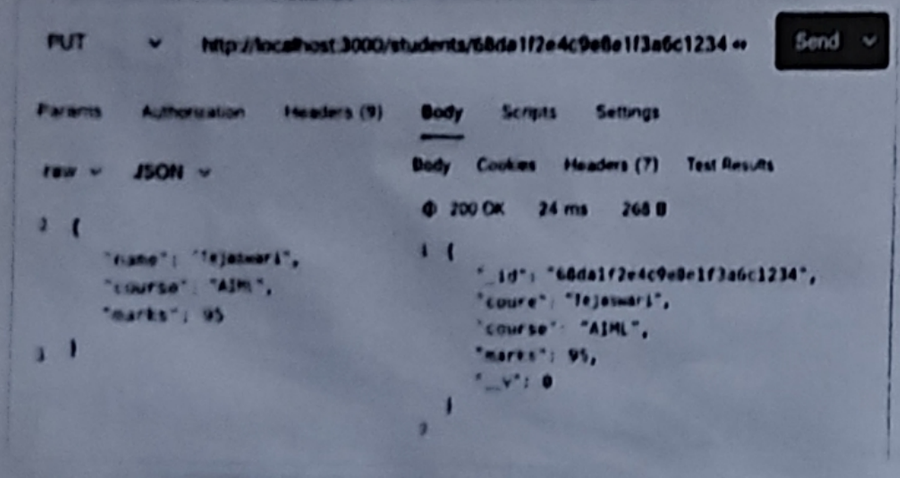
3. Create (POST) – Add Student



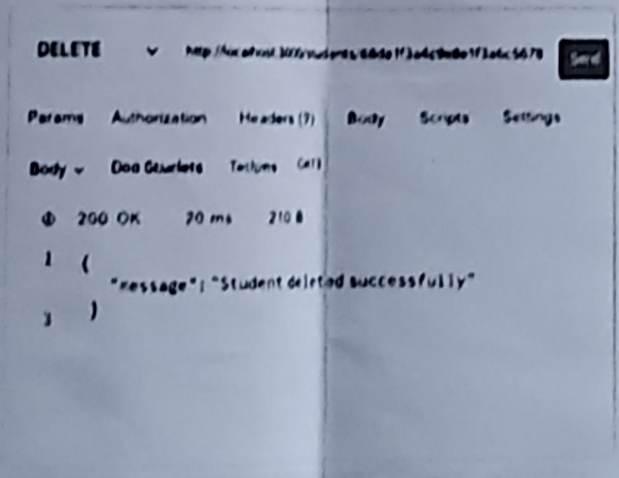
4. Read (GET) – Get All Students



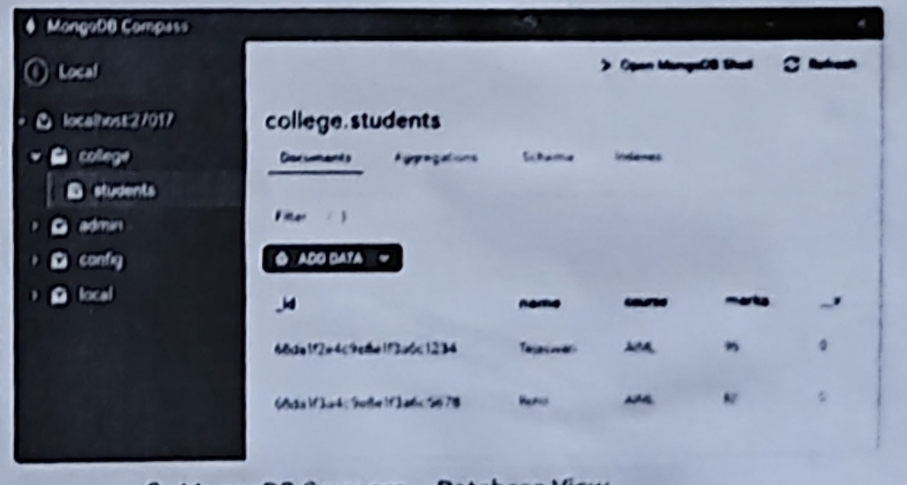
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View